IN3067/INM713 Semantic Web Technologies and Knowledge Graphs

Laboratory 5: Ontology Alignment

Ernesto Jiménez-Ruiz

Academic course: 2025-2026
Updated: February 9, 2026

Contents

1 Git Repositories **2**

2 Support Codes and Datasets **2**

3 Ontology Alignment Evaluation **3**

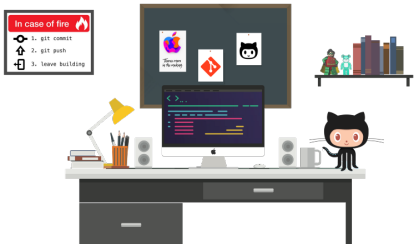
4 Tasks **3**

1 Git Repositories

Support codes for the laboratory sessions are available in *GitHub*. There are two repositories, one in Python and another in Java:

`https://github.com/city-knowledge-graphs`

For Java developers we use maven to deal with dependencies (see pom file). For Python developers there is always a `requirements.txt` within each lab folder with possibly new dependencies. It is recommended to use environments, but it is not strictly necessary.



2 Support Codes and Datasets

The GitHub repositories include support codes for this lab session, including the ontologies (owl format) and reference alignments (ttl format). The reference alignment files can also serve as an example of how your computed mapping should look.

In **Python** we will rely on the Owlready API (`https://owlready2.readthedocs.io/`)to load and process the ontologies. In **Java** we will continue with the Jena API (`https://jena.apache.org/documentation/ontology/`).

Support codes. I have added to the GiHub repositories the following support codes:

Getting the labels. The support scripts in Python (`AccessEntityLabels.py`) and Java (`AccessEntityLabels.java`) load an ontology, iterate over the classes and access the lexical information of the classes (*i.e.*, name/labels available as part of the URI or via the `rdfs:label` annotation).

String Similarity. String similarity will be useful to compare cell values to the label of KG candidates. In Python (see `lexical_similarity.py`) we use the `python-Levenshtein` library (see `requirements.txt`) and the ISub[1] method in `isub.py`. In Java (see `LexicalSimilarity.java`) we use the Apache Commons Lexical Similarity library[2] and the ISub class (`I_Sub.java`).

[1] A String Metric for Ontology Alignment. ISWC 2005: `http://manolito.image.ece.ntua.gr/papers/378.pdf`

[2] `https://commons.apache.org/proper/commons-text/apidocs/org/apache/commons/text/similarity/`

Precision and Recall. The reference scripts `CompareWithReference.py` and `CompareWithReference.java` compute Precision and Recall given as input the system computed mappings and the reference mappings for the corresponding matching task.[3] For Python developers a Jupyter notebook is also provided.

Note that not all matching tasks have a reference alignment, especially those coming from real-world applications. As described below, reference alignments are required for evaluation campaigns like the OAEI.

3 Ontology Alignment Evaluation

The Ontology Alignment Evaluation Initiative[4] (OAEI) is an international effort for the systematic evaluation of ontology alignment systems that has been running since 2004. The OAEI provides ontologies and reference alignment to drive a controlled evaluation among ontology alignment systems. In this lab session we are going to use two datasets from the OAEI.

Conference. This dataset aligns 7 ontologies from the same domain (conference organization): Cmt, ConfTool, Edas, Ekaw, Iasted, Sigkdd, Sofsem.

Anatomy. This evaluation dataset consists of finding an alignment between the Adult Mouse Anatomy (mouse) and a part of the NCI Thesaurus describing the human anatomy (human).

4 Tasks

Task 5.0 Basic ontology alignment can be reduced to comparing two list of elements.

1. Try to create a function that given two lists of elements, returns the list of elements in common (*i.e.*, equivalent). For example, given:
   `listA = ["pizza", "tomato sauce", "pepperoni", "restaurant"]`
   and
   `listB = ["pizza", "tomato", "peperone", "restaurant"]`
   return the `listC = ["pizza", "restaurant"]`.

2. Same as above but returning a list of "similar" elements. Recall the lexical similarity tools we used a few weeks back.

On the ontology setting you need to compare list of entities, and for each entity you know their URI and their label(s).

Task 5.1 Design and implement a (simple) lexical matcher that loads two ontologies and finds equivalence correspondences among their entities. Save the alignments as RDF triples in turtle format. For example:

[3] `https://en.wikipedia.org/wiki/Precision_and_recall`
[4] OAEI: `http://oaei.ontologymatching.org/`

```
cmt:Conference owl:equivalentClass confOf:Conference .
cmt:title owl:equivalentProperty confOf:hasTitle .
```

Tip 1: *align classes against classes and properties against properties. If there are instances, one should also target instances against instances using* `owl:sameAs` *(e.g.,* `city:ernesto owl:sameAs wd:ernesto`*.*
Tip 2: *Use lexical similarity and a threshold to decide which mappings are good enough to be added as output.*
Tip 3: *In previous laboratory sessions, among other things, we learnt how to create RDF triples.*

Task 5.2 Apply your algorithm over the following pairs of ontologies from the OAEI's conference track:

- cmt.owl - ekaw.owl
- cmt.owl - confOf.owl
- confOf.owl - ekaw.owl

Give suitable names to the mapping files (*e.g.*, `ernesto-cmt-ekaw.ttl`). Use the provided script and reference alignments to compute the precision and recall of the mappings computed by your system.

Task 5.3 Apply your algorithm over the OAEI's anatomy track and get the precision and recall of your mappings. Note that, unlike in the conference track, the ontologies in the anatomy track are relatively large and contain $2,744$ classes (mouse.owl) and $3,304$ classes (human.owl). If your algorithm computes a pair-wise comparison, this will imply $> 9,000,000$ lexical comparisons.[5]

Task 5.4 (Optional) Create an algorithm that *(i)* is scalable for large ontologies (*e.g.*, uses a lexical index) and *(ii)* involves not only the labels of the entities but also the structure of the ontology. This task gets closer to the *Coursework Part 2* objectives.

[5]LogMap (`https://github.com/ernestojimenezruiz/logmap-matcher`) is a state-of-the-art system that can match the ontologies in the anatomy track in 8s while producing competitive results. Its simple lexical variant LogMapLt takes only 2s. Results: `http://oaei.ontologymatching.org/2025/results/anatomy/`